Laboratório de Concorrência · Aula 10

VITOR HUGO ASSIS DA SILVA · RA 241400232 · Turma ESOFT6B
Gerado em 04/11/2026

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação é quando uma atividade depende de outra para avançar: um garçom espera o cozinheiro finalizar o prato antes de servi-lo. Competição ocorre quando várias pessoas precisam do mesmo recurso limitado: duas pessoas querem usar o único banheiro, mas uma deve esperar a outra sair.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

O total ideal é 2.000.000. Como as threads alteram o mesmo contador sem proteção, minha previsão é que a saída possa ficar abaixo desse número.

1ª execução (n = 1.000.000) · exemplo fornecido

1024774

2ª execução (n = 1.000.000) · exemplo fornecido

1040940

3ª execução (n = 1.000.000) · exemplo fornecido

1019841

Saída de referência do programa

```
esperado: 2000000
obtido: 1024774
Com n = 1000: 2000 nas duas execuções.

esperado: 2000000
obtido: 1019841
Com n = 1000: 2000 nas duas execuções.
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não. Nas três execuções registradas, o valor ficou abaixo de 2.000.000, e os três totais foram diferentes. Cada execução pode intercalar as leituras e escritas das threads de outro modo; por isso, a quantidade de incrementos perdidos varia.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Imagine o contador em 10. A lê 10; antes de gravar, B também lê 10. A calcula 11 e grava; B calcula 11 a partir da leitura antiga e grava 11 novamente. Embora tenham ocorrido dois incrementos, o contador aumentou só uma vez.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

No exemplo com 1.000 incrementos por thread, apareceram 2.000. A execução curta reduziu a chance de a falha se manifestar, mas não a eliminou. Um teste pequeno pode passar e, sob maior carga em produção, a mesma corrida pode reaparecer de forma intermitente.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8
BAAABB → 6
AABBBA → 4
BBAAAB → 6
ABBAAB → 6
BBBAAA → 7

Saída de referência do programa

```
AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 interleavings possíveis das três etapas de A e das três de B, preservando a ordem interna de cada uma. O simulador apresentou quatro totais: 4, 6, 7 e 8. Apenas AAABBB resultou em 8 no modelo mostrado. Os totais 4 e 6 aparecem em nove ordens cada; 7 e 8, em uma ordem cada.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Em AABBBA, A lê 3 e calcula 4, mas ainda não grava. B lê o valor antigo, calcula 6 e o escreve. Por último, A grava o 4 que havia calculado. A gravação final de A substitui o resultado de B; a operação de B foi executada, mas seu efeito se perdeu.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Espero que a versão sem sincronização possa terminar mais rápido, porém com resultado incorreto. As versões com synchronized e AtomicInteger devem preservar o total esperado. A velocidade relativa depende da máquina e da contenção.

Saída de referência do programa

```
esperado:          2000000
sem sincronizacao: 1019461 em 44 ms
synchronized:      2000000 em 108 ms
AtomicInteger:     2000000 em 26 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

No exemplo, synchronized e AtomicInteger chegaram a 2.000.000; a versão sem sincronização não chegou. O controle de acesso tem custo: synchronized pode fazer uma thread esperar a outra, e uma atualização atômica também exige coordenação. Nesta medição específica, AtomicInteger foi mais rápido que as outras versões; não dá para generalizar isso a partir de uma única execução.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Cada AtomicInteger protege a atualização individual de um saldo, mas a transferência envolve débito e crédito como uma unidade. Outra operação poderia enxergar o dinheiro retirado antes de ele ser depositado. É necessário garantir atomicidade para a transferência inteira, por exemplo com uma transação no banco ou bloqueios nas duas contas adquiridos sempre na mesma ordem.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Se A ficar com X enquanto espera Y e B ficar com Y enquanto espera X, nenhuma delas chega ao fim; portanto, não espero ver as mensagens de término.

Saídas de referência (antes e depois da correção)

```
IMPASSE: 2 threads presas
Saída depois da correção (B pega X e depois Y):
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A segurou X e aguardou Y; B segurou Y e aguardou X. Como cada thread retém o recurso necessário à outra, forma-se uma espera circular e nenhuma consegue prosseguir.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Quando ambas pegam X antes de Y, não há situação em que uma segura Y e espera X. A que obtiver X primeiro pode concluir o acesso e liberar os recursos. Um sistema de transferências bancárias pode aplicar essa regra bloqueando primeiro a conta de menor identificador, independentemente do sentido da transferência.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Sem sobrecarga, estimaria aproximadamente 600 ms com um trabalhador, 200 ms com três e 100 ms com seis. Os tempos reais podem variar um pouco.

Tempo com 1 trabalhador · exemplo/estimativa

600 ms (valor esperado para 6 tarefas sequenciais)

Tempo com 3 trabalhadores · exemplo/estimativa

200 ms (valor registrado na saída de referência)

Tempo com 6 trabalhadores · exemplo/estimativa

100 ms (valor esperado com 6 trabalhadores)

Saída de referência do programa

```
trabalhador 2 fez a tarefa 3
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 1
trabalhador 3 fez a tarefa 6
trabalhador 2 fez a tarefa 4
trabalhador 1 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com um trabalhador, as seis tarefas de 100 ms são sequenciais: cerca de 600 ms. Com três, cada um executa duas tarefas: cerca de 200 ms. Com seis, as seis podem correr em paralelo: cerca de 100 ms. O canal distribui cada tarefa para um recebedor; se os trabalhadores não modificarem outro estado mutável compartilhado, não é necessária uma trava para essa distribuição.

Saídas de referência (antes e depois da troca)

```
Exit 1
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
        /box/main.go:9 +0x38
exit status 2

Depois da troca: 1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Em um canal sem buffer, enviar só termina quando existe um recebimento correspondente. A goroutine main tenta enviar e fica bloqueada antes de alcançar sua própria instrução de receber. Um buffer de capacidade 1 guarda essa mensagem temporariamente, permitindo que main avance e a leia depois.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acredito que o Rust recuse o primeiro código na compilação, pois a thread não pode manter um empréstimo de uma variável local que pode sair de escopo.

Saídas de referência (o erro e o resultado com move)

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is owned
by the current function

Com move, o valor exibido por main foi 0.
```

Saída de referência com compartilhamento por Arc e Mutex

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O inteiro implementa Copy. Ao usar move nesse exemplo, a thread recebe sua própria cópia do valor inicial. Ela altera essa cópia, enquanto o contador que permanece em main continua igual a 0; por isso main imprime 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Na prática em Java, a perda de incrementos foi percebida ao executar e comparar o resultado; o compilador não aponta essa corrida. Em Rust, o empréstimo inseguro foi recusado antes da execução. Isso evita certas falhas de acesso à memória, mas exige modelar posse e compartilhamento com recursos adequados, como Arc e Mutex. Ainda podem existir impasses e erros de lógica.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A ordem esperada é A, D, C e B: primeiro o código síncrono, depois a reação da Promise e, por fim, o timer.

Saída de referência do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A sequência registrada foi A, D, C, B. A e D pertencem ao fluxo síncrono. Depois dele, executa-se a microtarefa da Promise, que imprime C. B é um callback de timer e só pode rodar mais tarde; atraso 0 não significa execução imediata.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Os 10 ms apenas tornam o timer elegível para execução. O cálculo síncrono manteve o laço de eventos ocupado por cerca de 300 ms, adiando E. Num servidor Node.js, um trabalho pesado na mesma thread atrasa o atendimento de outras requisições; pode ser necessário dividi-lo ou executá-lo em workers.

Fechamento: Quem impede a corrida?

Java sem sincronização

Não há proteção: o contador compartilhado fica sujeito a corrida.

Java com synchronized

A região crítica é protegida pelo mecanismo synchronized usado pelo programador.

Go com canais

O canal coordena a entrega das mensagens; o desenho pode evitar a disputa por um mesmo dado mutável.

Rust com Arc e Mutex

O compilador exige compartilhamento seguro; Arc e Mutex permitem dividir o estado e bloquear seu acesso.

JavaScript

Callbacks no mesmo laço executam um de cada vez, mas a ordem assíncrona ainda pode causar erros de estado; workers e serviços externos também introduzem concorrência.

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Os dois pedidos podem ler "1 lugar disponível" antes de qualquer reserva ser confirmada e, assim, vender o mesmo assento duas vezes. Eu faria a verificação e a reserva numa única transação ou atualização condicional no banco, para que só um pedido consiga confirmar a compra.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O mais surpreendente foi perceber que proteger cada incremento individualmente não garante, por si só, a correção de uma operação maior, como uma transferência entre duas contas.

Fontes técnicas

- Oracle: Thread Interference e Atomic Access
- Go: Effective Go (Channels)
- Rust Book: Shared-State Concurrency
- Node.js: The Event Loop